

Pramaan — Evidence Pipeline & Team Plan

Workflow answer, scope decisions, and updated per-person tasks

Answers the "what's the correct workflow" question, sets three scope decisions, and assigns what's left. Short on purpose — read it once, keep it open while you work.

Design ground rule still applies. Light theme only. DESIGN.md is dead.

1. The pipeline — resolved

Two ideas got framed as alternatives that aren't actually in tension. Here's the real sequence, and where each concern lands.

#	Step	What happens	Why nothing gets lost
1	Acquire	Bit-for-bit forensic image of the whole storage device (.dd/.raw/.E01). Hashed (MD5+SHA-256). Never touched again.	This step *is* "preserve everything first." The raw image contains every byte — including structures we don't yet know matter. Nothing downstream can lose data that's still sitting in the image.
2	Identify	Signature-match the vendor/filesystem directly on the image bytes.	Read-only. No conversion happening yet.
3	Parse	Vendor-specific adapter walks the proprietary index (HIKBTREE, DHFS, GPT-like) directly on the raw image. Extracts: channel, start/end timestamp, allocation state (normal / recording / deleted), byte offset.	This is "the friend's idea" — read the original structure directly, no intermediate conversion. It's correct: converting before you've parsed the index would destroy the very thing you're trying to read.
4	Recover	For each index entry — including ones flagged deleted — carve the byte range out of the image.	Deleted-flagged entries are still inside the image; the index just says the slot was reused/cleared. If the underlying bytes weren't overwritten, they carve out cleanly. If they were overwritten, recovery is genuinely impossible — mark PARTIAL, never invent frames to fill the gap.
5	Unwrap	Strip the vendor's per-recording wrapper (DHAV frame headers, HIKBTREE data block, Honeywell NAL headers) to expose the raw H.264/H.265 elementary stream underneath.	Not conversion. Every vendor wraps standard H.264/H.265 in a proprietary envelope — unwrapping removes the envelope, the video bytes are untouched. Reversible: given the image + the recorded byte offset, you can always re-derive this.
6	Repackage	Put the unwrapped stream into a standard container (.mp4) via ffmpeg -c copy — a remux, not a transcode. Re-encode only as a last-resort fallback when a stream is too damaged to copy, and that fallback must be logged, never silent.	Remux copies bits into a new box; zero quality loss, near-instant. Transcode decodes and re-encodes — slow, lossy, and never the first choice for evidence.
7	Normalize timestamps	Correct the recorder's own clock against a reference offset, per channel.	Metadata-level. Doesn't touch video bytes.
8	Hash + analyze	Hash the derived MP4 (separate from the image's hash, both recorded and linked). Run AI/cross-camera analysis on the derived, playable copy.	This is the one place lossy operations legitimately happen (frame decode for detection) — because it's *analysis*, not evidence transformation. Results are always labeled investigative leads, never presented as the evidence itself.

#	Step	What happens	Why nothing gets lost
—	Custody + report	Every step above appends a hash-linked custody entry. Report cites both hashes (image + derived clip) for every artifact shown.	Runs alongside every step, not after them.

One line if someone asks "don't we lose metadata on conversion?" — No, because we never convert the evidence. We read metadata from the untouched original in step 3, and derive an additional, always-regenerable playable copy in steps 5–6. The original and the extracted metadata both stay intact regardless of what happens to the derived copy.

2. Three scope decisions

Initial input format. A forensic disk image (.dd / .raw / .E01) of the DVR/NVR's storage — that is the PS's actual ask ("create forensic images"). Already-exported video files (.mp4, .dav, .264...) are accepted as a secondary shortcut path — useful when someone hands you a clip that's already off the device — but they skip identify/recover entirely because there's no proprietary filesystem left to parse. Don't present the shortcut path as equivalent to the real pipeline in a demo.

Corrupted footage. Recover what's genuinely present. Missing/overwritten bytes get marked PARTIAL — never silently patched with guessed or interpolated content. This is already the pattern in the codebase (recovery_status); keep it that way everywhere.

Upscaling / image enhancement — out of scope, on purpose. Not in the PS. More importantly: AI super-resolution on evidence is a real liability, not a feature — a generative model fills in pixels that were never captured, which is exactly the kind of thing that gets forensic video analysis thrown out (this is standard guidance in the field, not a Pramaan-specific opinion). If it comes up: Pramaan is one focused tool in a forensic workstation, the same way DVR Examiner and UFS Explorer are — recovery, parsing, and hashing here; hand off to a dedicated video-enhancement tool downstream if a case genuinely needs it. Saying this plainly is a stronger answer than building a half-safe upscaler.

3. Where things stand — spot-checked today, not a full re-audit

Area	Status	How I know
Cross-camera trace, Findings proximity flags	Shipped	Built and merged this session (Navneet).
Live devices	Removed entirely	Confirmed out of PS scope; deleted frontend/backend/tests/CI job. Drop it from every plan, script, and demo checklist.
Hikvision parser (hikvision_fs.py)	Solid	Cites 2 peer-reviewed papers + 1 corroborating OSS project; tracks deleted as a first-class state.
Hikvision recovery on the current lab image	Bug, reproduced today	lab_hikvision_fs.img identifies correctly (HIKBTREE match, 0.6 confidence) but recover returns 0 segments. Not yet root-caused.
Dahua recovery on the real .dav	Works	Ran it today end to end: acquire -> identify (0.98 confidence) -> recover -> 1 segment recovered -> playable.
Dahua DHFS parsing	Still carve-only	dahua_dhfs.py still scans for the DHAV frame magic and treats a 4KB gap as "deleted" — it never reads the DHFS index/allocation table.
Recovery page deleted column/filter	Present	deletedOnly toggle + allocation glyphs exist in CaseRecoverPage.tsx.
Ranged export (-ss/-c copy)	Verifies before trusting	Checks return code + file size, falls back to re-encode on failure — the old silent-empty-file bug looks fixed.
Report HTML preview	Still dark-themed	reporting.py still sets background:#0a0e1a — the one place the rejected dark UI is still live.

Area	Status	How I know
Job log	Still recovery-only	Only filters job.kind === "recovery" — analytics/export/import jobs still invisible.
Settings	"Parser sanity check" panel still present	Still shipped as a product feature, not moved to CI-only.
Evidence catalog	"Lab specimen" category still present	Still a real filter option in the UI.

4. Who does what

Navneet is on cross-camera trace only — already shipped, still refining. Not assigned anything below.

Aravind — Hikvision recovery

- Root-cause the 0-segments bug on lab_hikvision_fs.img (identification works, recovery doesn't — start at HikvisionAdapter._scan / list_recordings).
- If a real (not lab-built) Hikvision disk image can be sourced, swap it in and re-verify every offset in docs/reference/hikvision_fs.md against it.
- **Done when:** the lab image (or a real one) recovers a non-zero, correct count of recordings, at least one marked deleted, with a regression test.

Aslam — Dahua filesystem + the unwrap/remux pipeline

- Replace the DHAV magic-carve with real DHFS index parsing (partition marker DHFS4.1) — enumerate recordings from the index instead of scanning for frame headers. Keep the magic-scan as an explicit carve_fallback for damaged indexes.
- Entries whose byte range falls outside any allocated index entry -> deleted (unallocated extent), replacing the current 4KB-gap heuristic.
- Audit-round-5 already tightened the ranged-export ffmpeg path — re-verify it against a Dahua recording specifically (today's test only exercised the full-segment path).
- **Done when:** Dahua recovery reports a real deleted count sourced from the index, not a gap heuristic.

Roshan — Timeline & playback, multi-channel

- Confirm the per-channel grouping in timeline.py actually drives a **shared absolute axis** in TimelineView.tsx with one scrub moving every lane (this existed as a gap in the last audit — re-verify it's actually wired end to end, not just present in the backend).
- Deleted segments get a distinct lane color + legend + "deleted only" toggle, fed from the real allocation_state Aravind/Aslam now emit.
- **Done when:** a 2-channel case shows both lanes on one absolute axis, deleted segments visually distinct, one scrub moves both.

Jai Pranav — Report, Job log, Settings, Evidence catalog, integration

- **Report:** restyle reporting.py's generated HTML off the dark palette (#0a0e1a -> white/near-black/one blue accent, matching every other screen). Last place the rejected dark UI survives.
- **Job log:** add a job-kind column; stop filtering to recovery only — analytics, export, and cross-camera jobs need to show up too.
- **Settings:** remove the "Parser sanity check" panel and its backend; those checks belong in CI (pytest), not the product.
- **Evidence catalog:** remove the "Lab specimen" category/filter/icon.
- **Integration:** update (or retire) scripts/demo/mvp.sh — it still assumes Live devices exists as a demo step; drop it. Re-run the demo script end to end on the real Dahua .dav and confirm every step in Section 1 above actually

happens on screen.

- **Done when:** report preview is light-themed, job log shows every kind, Settings/Evidence catalog have no fake surfaces left, and the demo script runs clean top to bottom.
-

■ 5. Before calling anything done

- `python -m pytest engine/tests -q green`.
- Frontend typecheck, build, and format check clean on your changed files (`npm run build`, `npx tsc`, `npm run format:check`).
- Your "done when" line above, demonstrated on real evidence — not a fixture, not your first try.